

补充模拟卷 03：中间代码表示补缺

A 卷题目

一、四元式、三元式、间接三元式（25 分）

将表达式：

$$x = (a + b) * (c - d) + e$$

翻译为：

1. 普通三地址码。
2. 四元式。
3. 三元式。
4. 间接三元式。

二、函数调用与参数传递（15 分）

将下面语句翻译为三地址码：

$$y = f(a, b + c)$$

要求体现 param 和 call。

三、数组访问与地址计算（20 分）

假设 int 宽度为 4，将下面语句翻译为三地址码：

$$\begin{aligned} val &= a[i] \\ a[i] &= y \end{aligned}$$

要求体现下标地址计算。

四、fall-through 与减少冗余跳转（20 分）

为下面语句生成三地址码，尽量利用顺序执行减少冗余 goto：

$$\begin{aligned} \text{if } (x < 100 \parallel x > 200 \ \&\& \ x \neq y) \\ \quad x &= 0; \end{aligned}$$

说明 fall-through 的含义。

五、do-while、break、continue（20 分）

1. 为下面语句生成三地址码：

$$\begin{aligned} \text{do } \{ \\ \quad a &= a + 1; \\ \} \text{ while } (a < 10 \ \&\& \ b > 0); \end{aligned}$$

2. 说明 break 和 continue 在循环中的跳转目标。

B 按考试标准重写答案

一、四元式、三元式、间接三元式答案

1. 普通三地址码

表达式：

$$x = (a + b) * (c - d) + e$$

先计算括号内的两个子表达式，再乘法，再加 e，最后赋给 x：

步骤	子表达式	临时变量
1	$a + b$	t1
2	$c - d$	t2
3	$t1 * t2$	t3
4	$t3 + e$	t4
5	赋值给 x	x

三地址码：

```
t1 = a + b
t2 = c - d
t3 = t1 * t2
t4 = t3 + e
x = t4
```

2. 四元式

四元式格式为：

(op, arg1, arg2, result)

序号	op	arg1	arg2	result
1	+	a	b	t1
2	-	c	d	t2
3	*	t1	t2	t3
4	+	t3	e	t4
5	=	t4	-	x

3. 三元式

三元式格式为：

(op, arg1, arg2)

结果不再显式命名为 t1、t2，而是用三元式序号引用。

序号	op	arg1	arg2	含义
0	+	a	b	(a+b)
1	-	c	d	(c-d)
2	*	(0)	(1)	(a+b)*(c-d)
3	+	(2)	e	(a+b)*(c-d)+e
4	=	(3)	x	x=(3)

写成列表：

0: (+, a, b)

- 1: (-, c, d)
- 2: (*, (0), (1))
- 3: (+, (2), e)
- 4: (=, (3), x)

4. 间接三元式

间接三元式由“三元式表”和“指针表”组成。三元式表保持不变：

三元式序号	op	arg1	arg2
0	+	a	b
1	-	c	d
2	*	(0)	(1)
3	+	(2)	e
4	=	(3)	x

指针表决定实际执行顺序：

指针项	指向三元式	当前执行内容
p0	0	a+b
p1	1	c-d
p2	2	(0)*(1)
p3	3	(2)+e
p4	4	x=(3)

间接三元式的优点是：优化时若要调整执行顺序，可主要调整指针表，而不必移动三元式本身。

二、函数调用与参数传递答案

语句：

```
y = f(a, b + c)
```

先计算非简单实参 $b + c$ ，再传参并调用函数。三地址码为：

```
t1 = b + c
param a
param t1
t2 = call f, 2
y = t2
```

逐行说明：

行	作用
t1 = b + c	先求第二个实参表达式
param a	传入第一个实参
param t1	传入第二个实参
t2 = call f, 2	调用 f，实参数量为 2，返回值放入 t2
y = t2	把返回值赋给 y

若课堂约定参数从右向左压栈，则 param 顺序可按约定调整；本答案采用题面常见的从左到右传参写法。

三、数组访问与地址计算答案

题目给定 int 宽度为 4。若数组 a 的基地址记为 a，则：

$a[i]$ 的地址 = $a + i * 4$

1. `val = a[i]`

步骤	计算内容	三地址码
1	计算字节偏移	$t1 = i * 4$
2	计算元素地址	$t2 = a + t1$
3	从地址取值	$val = *t2$

完整代码：

```
t1 = i * 4
t2 = a + t1
val = *t2
```

2. `a[i] = y`

步骤	计算内容	三地址码
1	计算字节偏移	$t3 = i * 4$
2	计算元素地址	$t4 = a + t3$
3	写入地址	$*t4 = y$

完整代码：

```
t3 = i * 4
t4 = a + t3
*t4 = y
```

注意：`val = a[i]` 是取右值，需要从地址读；`a[i] = y` 是写左值，需要先算出地址再写入。

四、fall-through 与减少冗余跳转答案

1. 条件结构分析

语句：

```
if (x < 100 || x > 200 && x != y)
    x = 0;
```

&& 优先级高于 ||，所以条件等价于：

```
(x < 100) || ((x > 200) && (x != y))
```

短路逻辑如下：

子条件	为真时	为假时
$x < 100$	整体为真，执行 then	继续判断 $x > 200$
$x > 200$	继续判断 $x != y$	整体为假，跳到出口
$x != y$	整体为真，执行 then	整体为假，跳到出口

2. 利用 fall-through 的三地址码

把 then 分支放在条件判断之后，让最后一个真路径自然落入 then 分支：

```
if x < 100 goto L_then
if x <= 200 goto L_next
if x == y goto L_next

L_then:
x = 0

L_next:
```

逐行解释：

代码	说明		
if x < 100 goto L_then	左侧 `		` 为真，直接执行 then
if x <= 200 goto L_next	x > 200 为假，整体条件为假		
if x == y goto L_next	x != y 为假，整体条件为假		
顺序落入 L_then	说明 x > 200 且 x != y 都为真		

3. fall-through 含义

fall-through 指“不生成显式跳转指令，而让控制流按代码顺序自然进入下一条指令”。本题中第三个判断之后没有写：

```
goto L_then
```

而是让满足条件的路径直接顺序执行到 L_then，减少了一条冗余跳转。

五、do-while、break、continue 答案

1. do-while 三地址码

源程序：

```
do {
    a = a + 1;
} while (a < 10 && b > 0);
```

do-while 是后测试循环，循环体至少执行一次。条件是：

```
(a < 10) && (b > 0)
```

&& 的短路规则是：第一个条件为假时，不再检查第二个条件，直接退出循环。

三地址码：

```
L_body:
t1 = a + 1
a = t1

L_test:
if a < 10 goto L_check_b
goto L_exit

L_check_b:
if b > 0 goto L_body
goto L_exit

L_exit:
```

流程说明：

标签	作用
L_body	循环体开始位置
L_test	do-while 底部条件检查位置
L_check_b	只有 a < 10 为真时才检查 b > 0
L_exit	循环出口

2. `break` 和 `continue` 的跳转目标

在循环翻译中，break 和 continue 不是普通顺序执行语句，它们直接改变控制流：

语句	while 循环中的目标	do-while 循环中的目标
break	跳到循环出口	跳到循环出口
continue	跳到循环头的条件判断	跳到底部条件判断

对应本题的 do-while：

```
break    -> goto L_exit
continue -> goto L_test
```

原因是 continue 表示结束本轮循环并进入下一轮判断；对 do-while，下一轮判断发生在循环体之后的测试位置。